CS 530 Lab Notebook

Lab 4

Name: Jarren Calizo
OdinID: calizo
Date: October 26, 2025

Table of Contents

Lab Overview 2
Lab 4.1g nginx Compute Engine Guestbook 3
Step 1: Overview 3
Step 2: Compute Engine VM Instance 3
Step 3: Generate a DNS Name for VM 3
Step 4: Checkout Code 4
Step 5: Examine Code 5
Step 6: Install the Application 5
1. Take a screenshot of the site along with its Let's Encrypt certificate and include it in your lab notebook. 6
Step 7: Clean Up 6
Lab 4.2g Docker Guestbook 7
Step 1: Containers 7
Step 2: Version 1: Ubuntu 7
Step 3: Build and run the Ubuntu-based container 8
1. Take a screenshot of the results for your lab notebook. 8
Step 4: Docker Commands 9
Step 5: Docker Hub Ubuntu 10
Step 6: Running from Docker Hub 10
1. Run the image directly from Docker Hub and show a screenshot of the output of the command in your lab notebook. 11
2. Take a screenshot of the container image and its size for your lab notebook. 11
3. What layer adds the most to the container image? How much does it add? 12
Step 7: Version 2: Alpine 12
1. Take a screenshot of the image generated and its size for your lab notebook. How much smaller is the image than the Ubuntu one? 13
Step 8. Build and Run the Alpine-Based Container 14
1. Show the output of this command in a screenshot for your lab notebook. What might have happened? 14
2. Take a screenshot of the output of each. 15
Step 9: Docker Hub Alpine 15
1. Take a screenshot of the container image and its size. 16
Step 10: Compute Engine Ubuntu VM Deployment 17
1. Take a screenshot of the entry that includes the VM's external IP address for your lab notebook. 17
Step 11: Clean Up 18

Lab Overview

Lab 4 marked a pivotal transition in understanding the continuum between traditional virtual machine deployments and modern containerized cloud infrastructures. It was divided into two complementary parts: Lab 4.1g focused on deploying a full-stack web application (Guestbook) on a Google Compute Engine VM with nginx and HTTPS certification, while Lab 4.2g explored Docker containerization and multi-platform portability of that same application.

The first half of the lab illuminated the "lift-and-shift" paradigm - migrating an on-premise web service into the cloud. I began by provisioning a Compute Engine instance, configuring firewall rules for HTTP and HTTPS, and binding the VM to a dynamic DNS name through No-IP. The deployment process deepened my understanding of cloud networking fundamentals: DNS propagation, static IP associations, and the orchestration of reverse proxies via nginx. After cloning and inspecting the Flask Guestbook application, I installed it through a scripted automation that integrated gunicorn, systemd, and nginx templates. The final moment-securing the domain cs530calizo.hopto.org with a Let's Encrypt certificate-felt symbolic: a once-local app now accessible globally under HTTPS, its padlock icon a quiet emblem of trust.

The second half, Lab 4.2g, transitioned from virtual machines to containers, highlighting Docker’s precision and efficiency. Beginning with an Ubuntu-based image, I walked through the full container lifecycle-building, running, publishing to Docker Hub, and pulling remotely. Observing the image layers and pinpointing the largest contributor (system dependencies) revealed how layered caching streamlines deployment. Later, by swapping Ubuntu for Alpine Linux, the image footprint shrank from over 500 MB to barely 57 MB, without compromising the app’s functionality. This contrast between comprehensiveness and minimalism embodied the elegance of container design: same output, less overhead.

Finally, pushing the containerized Guestbook to Docker Hub and deploying it onto a Compute Engine VM closed the loop-proof that a single lightweight image could traverse from local development to public cloud execution seamlessly. Through these steps, I experienced firsthand how Docker unifies the software pipeline: code written once, built once, and deployed anywhere.

In essence, Lab 4 was more than an exercise in commands and configurations-it was a synthesis of systems thinking. It bridged static servers and dynamic clouds, manual setup and automated deployment, heavy VMs and featherlight containers. Watching the Guestbook respond under a secure global domain and later run identically through Docker reinforced a simple yet profound truth: the modern web thrives not just on infrastructure, but on adaptability.

Lab 4.1g nginx Compute Engine Guestbook

Step 1: Overview

This lab emulates the traditional "lift-and-shift" deployment model where a legacy web server is migrated to the cloud. By provisioning a Compute Engine VM, binding it to a DNS name, and layering on nginx with a Python/Flask Guestbook application, the exercise demonstrates the evolution from a static on-premise site to a secure HTTPS-enabled cloud-hosted service. This was the first step toward understanding how virtualized infrastructure, DNS routing, and certificate automation integrate in modern web deployment.

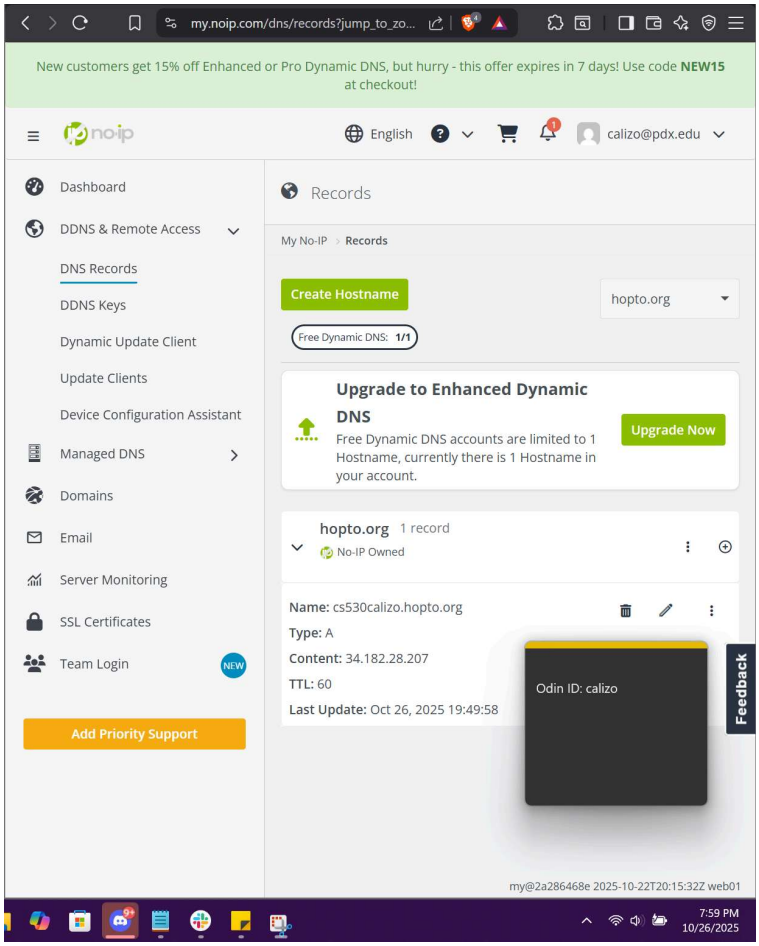
Step 2: Compute Engine VM Instance

Using Cloud Shell, I initiated firewall rules for both HTTP and HTTPS traffic to allow inbound web connections. After confirming existing rule warnings were harmless, I created the Compute Engine instance using the Ubuntu 22.04 LTS image and tagged it for web traffic.

The instance was provisioned in the us-west1-b zone as nginx-gb, outputting an external IP address that would serve as the public entry point for the Guestbook application.

Step 3: Generate a DNS Name for VM

To ensure user-friendly access without manually recalling the IP, I created a hostname through No-IP's dynamic DNS service. The hostname cs530calizo.hopto.org was registered and pointed to my external IP (34.182.28.207).



This binding provided a clean alias to the VM, bridging traditional DNS configuration with cloud-based addressing. Seeing the record reflect with TTL 60 confirmed proper propagation and readiness for HTTPS validation.

Step 4: Checkout Code

After establishing SSH access into the VM, I adjusted directory permissions to allow the web server process to traverse my home directory.

I then cloned the cs430-src repository and navigated into 03_nginx_gunicorn_certbot, the working directory containing the Flask Guestbook application and deployment scripts. The structure included systemd and nginx templates, the Python app files, and an automated install script. At this stage, the foundation for the web service was fully staged within the virtual machine.

Step 5: Examine Code

I reviewed the following key files:

- wsgi.py - a simple entry point importing app from app.py and exposing it for gunicorn.
- etc/systemd.template - a startup configuration ensuring the Flask app launches automatically on boot with proper environment paths.
- etc/nginx.template - defines routing for both dynamic content (through gunicorn socket) and static content (served directly by nginx).
- install.sh - the primary automation script, dynamically generating config files, creating a Python virtual environment, and integrating systemd + nginx with the domain.

Understanding these templates clarified how nginx proxies requests to gunicorn, which in turn serves the Flask Guestbook. The modularity mirrors production-grade deployments.

Step 6: Install the Application

Executing:

- sudo ./install.sh cs530calizo.hopto.org

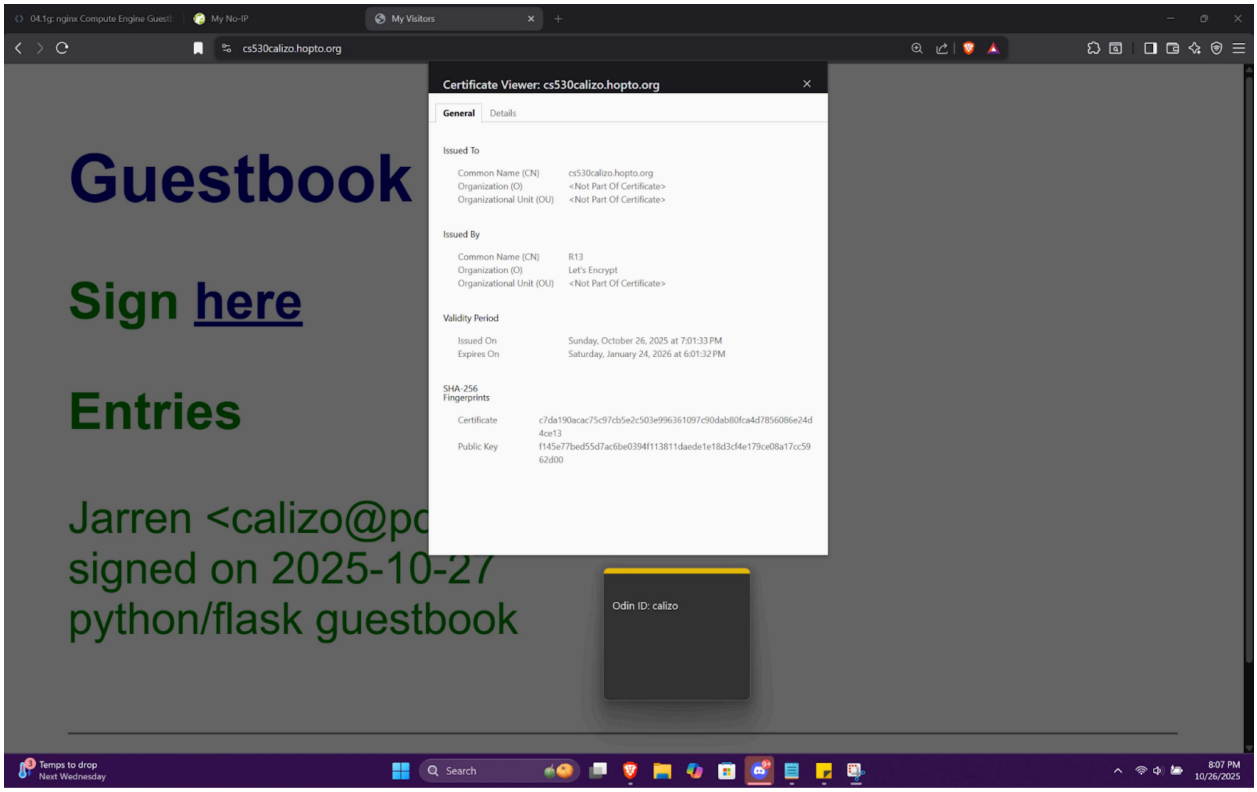
triggered the installation of dependencies, creation of the virtual environment, and auto-generation of configuration files. The site came online successfully when accessed via http://cs530calizo.hopto.org, displaying the Flask Guestbook interface.

Next, I secured it with a TLS certificate:

- sudo certbot --nginx -d cs530calizo.hopto.org -n -m calizo@pdx.edu --agree-tos --redirect

Upon completion, the site was reachable over HTTPS. Verifying the certificate revealed Let's Encrypt R13 as the issuer with validity from Oct 26, 2025 to Jan 24, 2026, confirming end-to-end SSL setup.

1. Take a screenshot of the site along with its Let's Encrypt certificate and include it in your lab notebook.

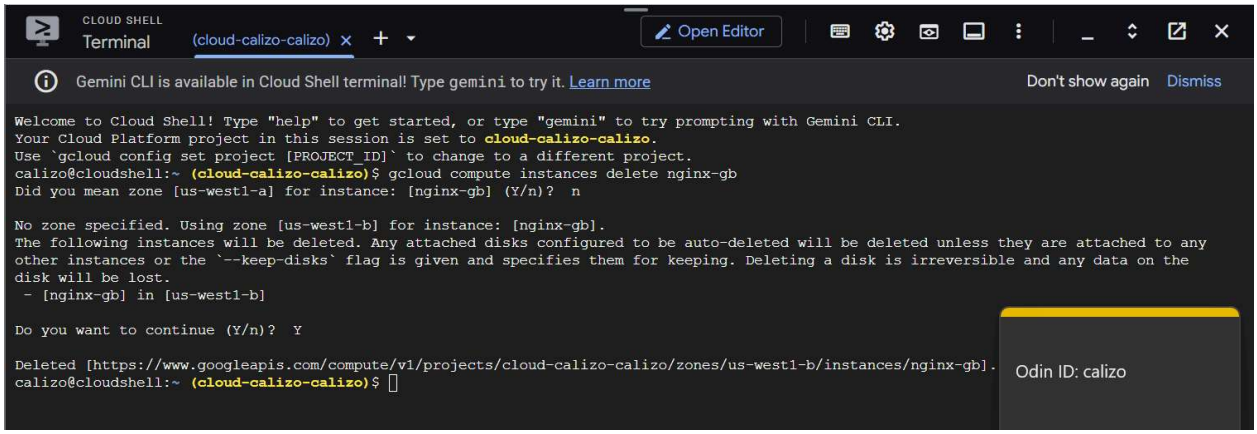


I then submitted a test Guestbook entry under my credentials, validating both frontend and backend integration.

Step 7: Clean Up

Finally, to prevent ongoing cloud resource billing, I deleted the instance from Cloud Shell with:

- gcloud compute instances delete nginx-gb



Confirming “Y” completed the teardown. The command output verified that all associated disks and configurations were removed, leaving no lingering resources. This wrapped up the lifecycle of the deployed web service - from provisioning and deployment to validation and cleanup - illustrating a complete DevOps-style flow.

Insight
This lab bridged the gap between theory and tangible cloud architecture. Standing up a full-stack application on a Compute Engine VM, binding it to a DNS name, and layering HTTPS with automation through certbot showcased how legacy architectures can evolve toward modern deployment practices. Seeing my custom domain securely render the Guestbook entry felt like a miniature milestone in understanding end-to-end web service provisioning.

Lab 4.2g Docker Guestbook

Step 1: Containers

Containers bridge the gap between operating system isolation and lightweight deployment. Unlike full virtual machines, which each emulate their own OS kernel, containers share the host kernel while keeping application environments separate and reproducible. For this lab, I SSHed into my course Compute Engine VM, cloned the course repository, and navigated into the directory containing the Guestbook container files. This set the stage for building a containerized version of the same Guestbook Flask app used in previous labs.

Step 2: Version 1: Ubuntu

I examined the provided Dockerfile.ubuntu and saw how it defined the build layers: starting from Ubuntu 20.04, updating packages, installing Python3-pip, copying the application into /app, installing dependencies via requirements.txt, and finally defining the entrypoint and command to launch app.py.

Before building, I edited the maintainer line to include my PSU email. This ensured the image metadata correctly reflected my identity as the image author. The file structure illustrated how Docker images stack their layers efficiently, minimizing redundant data when multiple containers share similar bases.

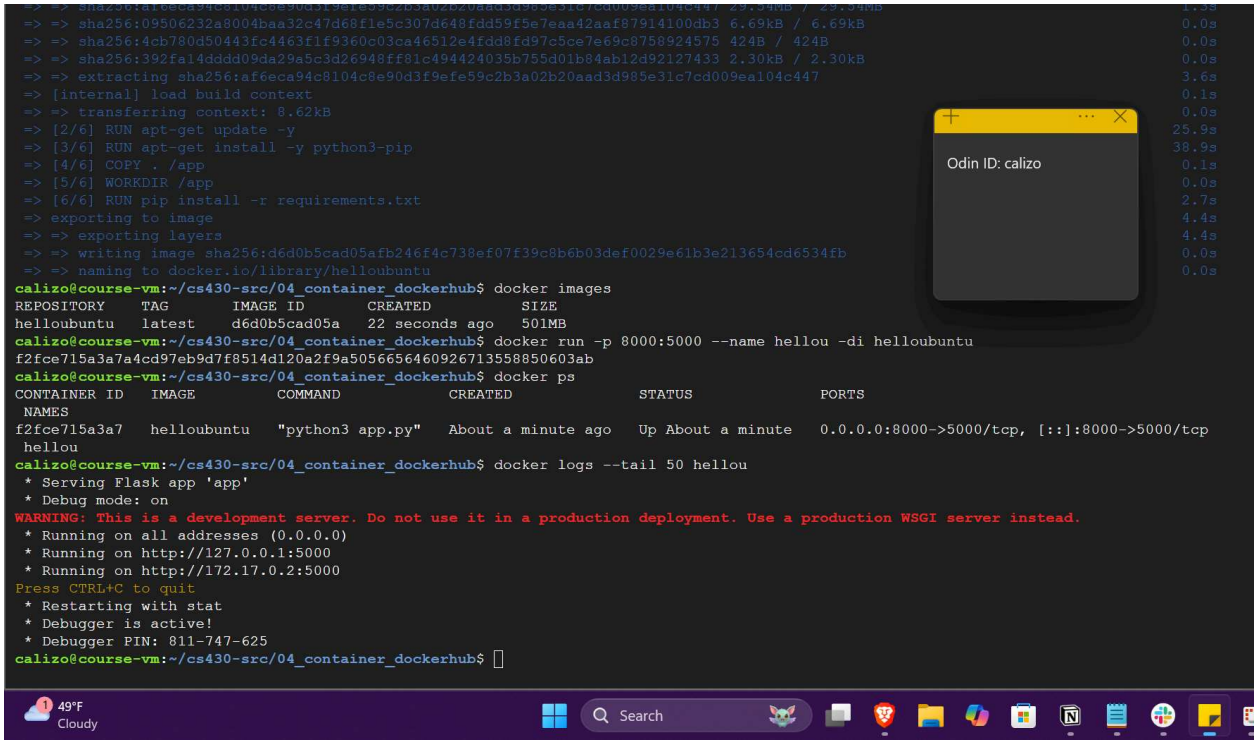
Step 3: Build and run the Ubuntu-based container

With Docker installed and permissions configured, I proceeded to build the image using:

- docker build -f Dockerfile.ubuntu -t helloubuntu .

The build process showed each layer being cached and exported, resulting in a 501MB image. Running docker images confirmed successful creation of helloubuntu, tagged as latest.

1. Take a screenshot of the results for your lab notebook.



Next, I launched the container:

- docker run -p 8000:5000 --name hellou -di helloubuntu

The container initialized in detached mode, mapping the host port 8000 to container port 5000. Running docker ps displayed hellou as active, with its exposed ports correctly linked.

To verify, I tailed the logs:

- docker logs --tail 50 hellou

The output confirmed that Flask's development server was running on 0.0.0.0:5000 and also accessible through 127.0.0.1:8000 from the host. The final message showed "Serving Flask app 'app'" with debugging enabled, confirming that the Guestbook container was operational and responding as expected.

This moment encapsulated the essence of containerization - deploying the same web application inside a self-contained runtime, isolated from the host system yet sharing its resources seamlessly.

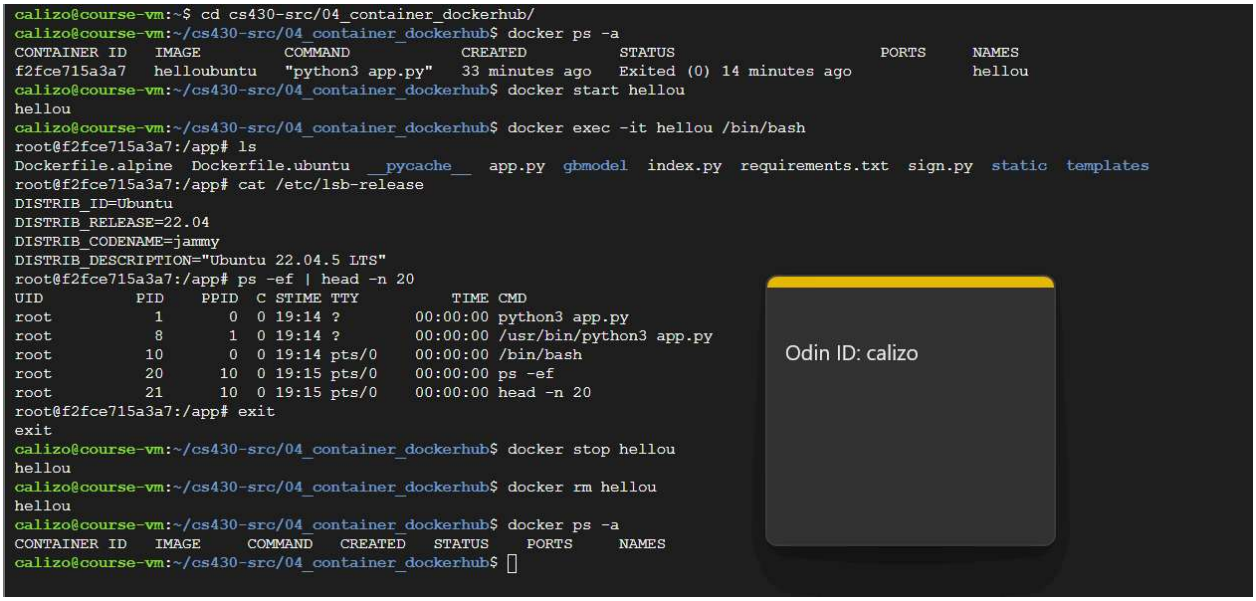
Step 4: Docker Commands

This step provided hands-on familiarity with Docker's core management workflow - inspecting, starting, stopping, and attaching to containers.

Using docker ps -a, I confirmed the hellou container instance and noted its exited status. Restarting it with docker start hellou reactivated the environment. Entering the container interactively via docker exec -it hellou /bin/bash, I verified the application's internal structure with ls, confirmed the base OS as Ubuntu 22.04 through /etc/lsb-release, and listed running processes with ps -ef.

After exploring, I exited the shell, then issued:

- docker stop hellou
- docker rm hellou



This removed the running container instance while leaving the helloubuntu image intact for reuse. The sequence reinforced Docker's separation between image blueprints and ephemeral container instances.

Step 5: Docker Hub Ubuntu

Next, I published the locally built image to Docker Hub for remote accessibility. After authenticating using:

- docker login

I tagged the image under my Docker Hub namespace:

- docker tag helloubuntu jarrenx/helloubuntu

and successfully pushed it to the repository:

- docker push jarrenx/helloubuntu

Each layer upload appeared as a hash sequence, confirming the push. Running docker images then verified both the local helloubuntu and its Docker Hub counterpart jarrenx/helloubuntu, both approximately 501MB in size. This mirrored how organizations sync internal container registries with public or private repositories for consistent deployment.

Step 6: Running from Docker Hub

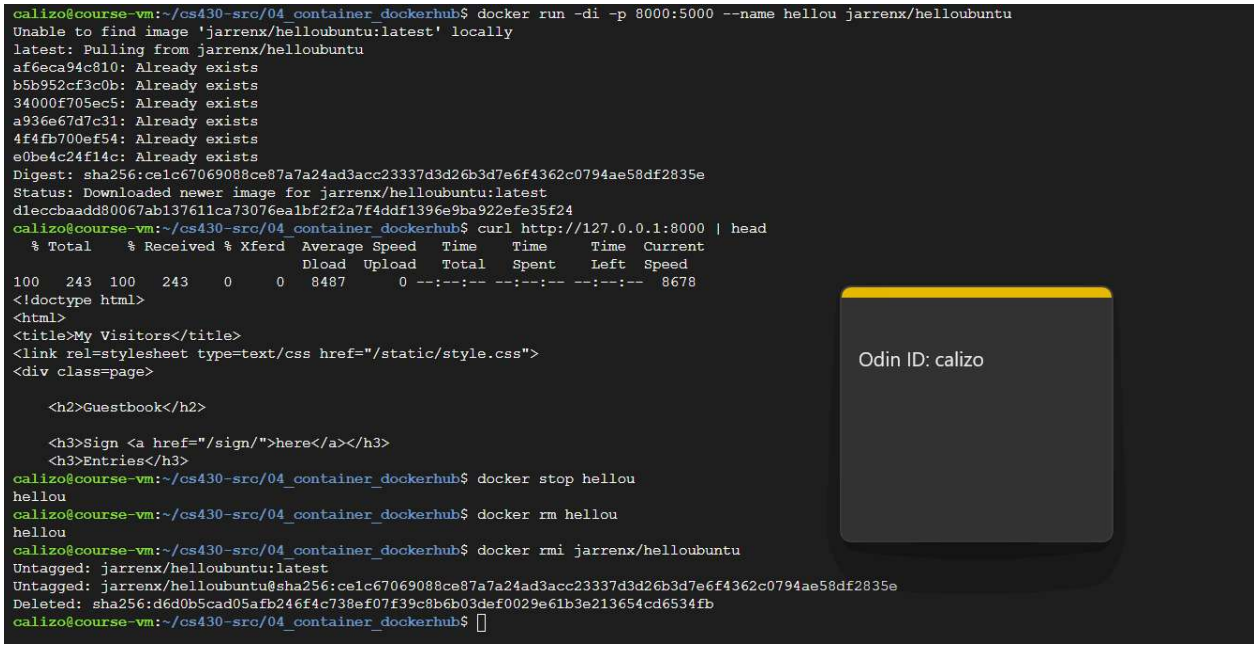
After confirming the push, I removed both local images to ensure the next container would be fetched directly from Docker Hub:

- docker rmi helloubuntu jarrenx/helloubuntu

Then, I ran the image remotely using:

- docker run -di -p 8000:5000 --name hellou jarrenx/helloubuntu

1. Run the image directly from Docker Hub and show a screenshot of the output of the command in your lab notebook.



Docker automatically pulled the layers from Docker Hub, showing incremental fetches for each cached component. Once complete, I verified functionality by testing the container endpoint:

- curl http://127.0.0.1:8000 | head

The response displayed the HTML structure of the Guestbook app, confirming it was running correctly through the pulled image.

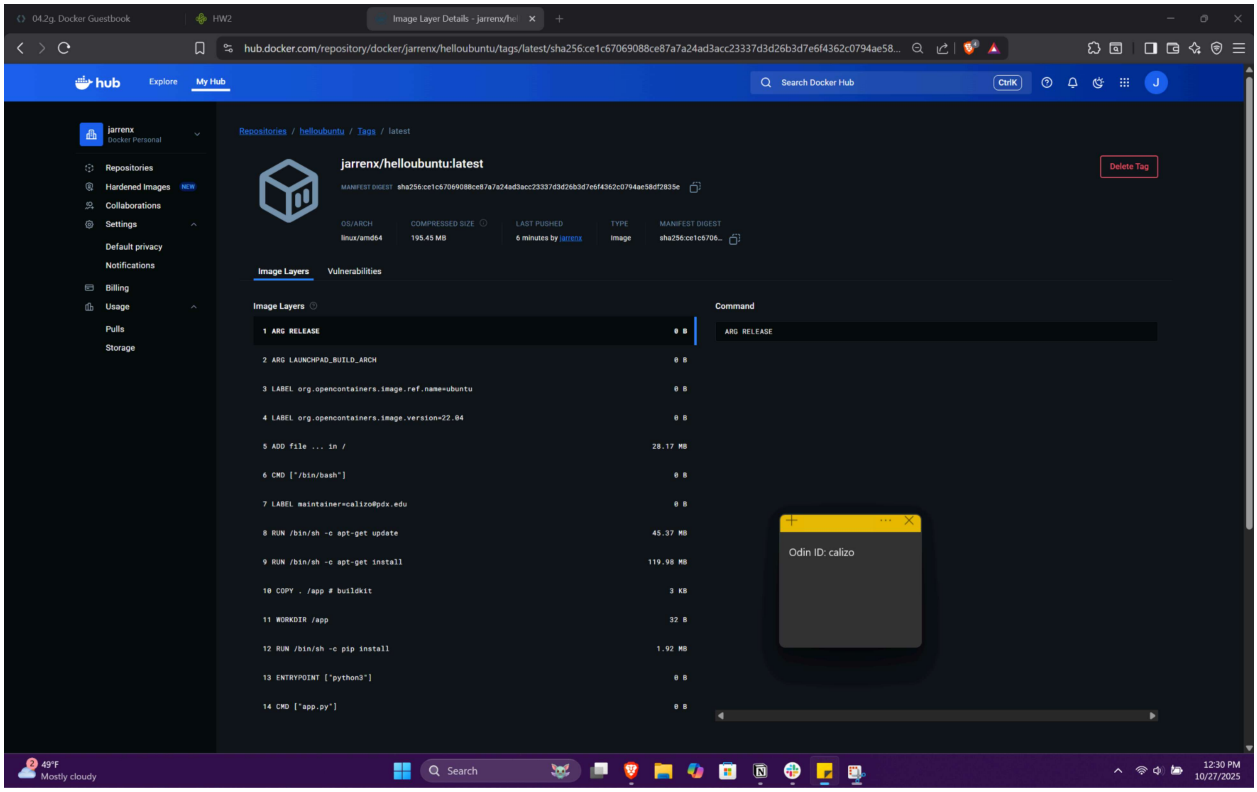
After testing, I cleaned up the environment:

- docker stop hellou
- docker rm hellou
- docker rmi jarrenx/helloubuntu

Then, I navigated to Docker Hub → jarrenx/helloubuntu → latest tag, viewing the image details and build layers.

2. Take a screenshot of the container image and its size for your lab notebook.

The compressed size shown was 195.45 MB.



3. What layer adds the most to the container image? How much does it add?

The layer corresponding to RUN /bin/sh -c apt-get install contributed the most, adding approximately 119.9 MB, which represents the majority of the system dependencies and Python packages required to run Flask and its libraries.

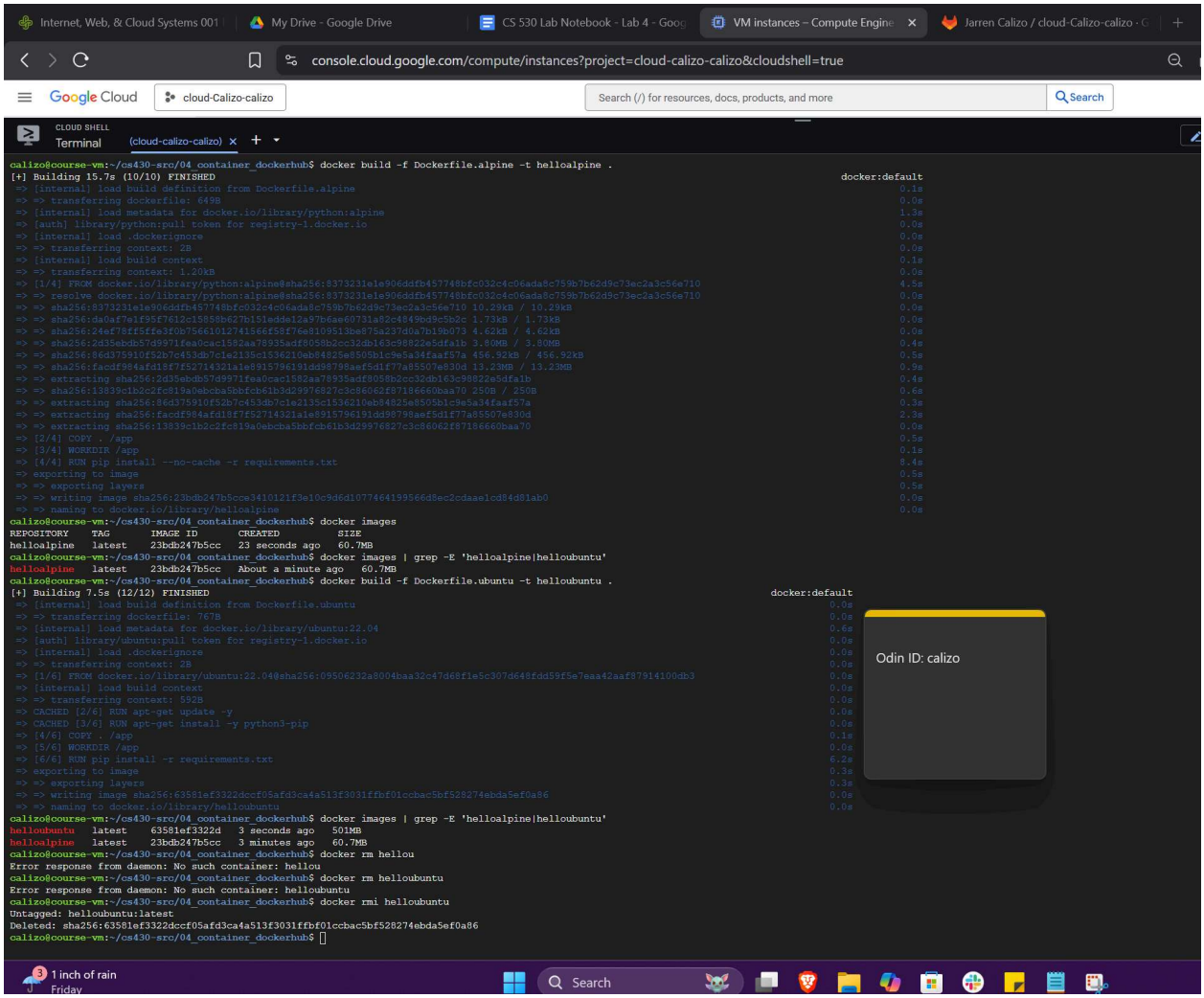
Insight

These steps tied together the practical life cycle of a container: from creation and exploration to publishing, pulling, and redeployment from a registry. Seeing my own image downloaded from Docker Hub and launched seamlessly validated the portability principle behind containerization - one image, identical performance, anywhere it runs.

Step 7: Version 2: Alpine

In this step, I shifted focus from the Ubuntu-based image to an Alpine-based one to emphasize efficiency and minimalism. Alpine is a lightweight Linux distribution that removes unnecessary runtime packages, making it a natural choice for container optimization. Inside the working directory cs430-src/04_container_dockerhub, I updated the maintainer field in Dockerfile.alpine to include my PSU email.

1. Take a screenshot of the image generated and its size for your lab notebook. How much smaller is the image than the Ubuntu one?



Building the new image with:

- docker build -f Dockerfile.alpine -t helloalpine .

successfully produced the helloalpine container image. Comparing image sizes via docker images, the Alpine image was 57.3MB, significantly smaller than the 501MB Ubuntu counterpart - nearly 443MB smaller.

This reduction highlights the practicality of base image selection in real-world deployments. Using Alpine decreases storage consumption and reduces startup latency without sacrificing application stability.

Step 8. Build and Run the Alpine-Based Container

After building the lightweight image, I launched the container:

- docker run -p 8000:5000 --name helloa -di helloalpine

Verifying functionality with:

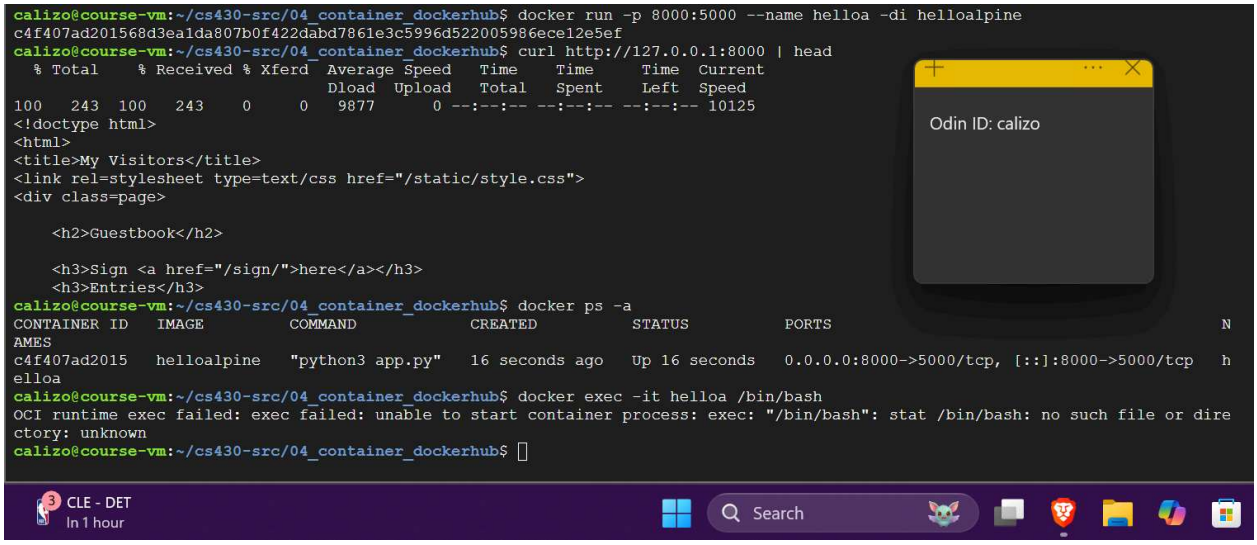
- curl http://127.0.0.1:8000 | head

confirmed that the Guestbook application rendered properly in the terminal, showing the expected HTML page structure.

To inspect the container internals, I ran:

- docker exec -it helloa /bin/bash

1. Show the output of this command in a screenshot for your lab notebook. What might have happened?



The command failed, displaying an error indicating /bin/bash could not be found. This occurred because Alpine uses BusyBox, which provides the simpler /bin/sh shell instead of bash.

Replacing the command with:

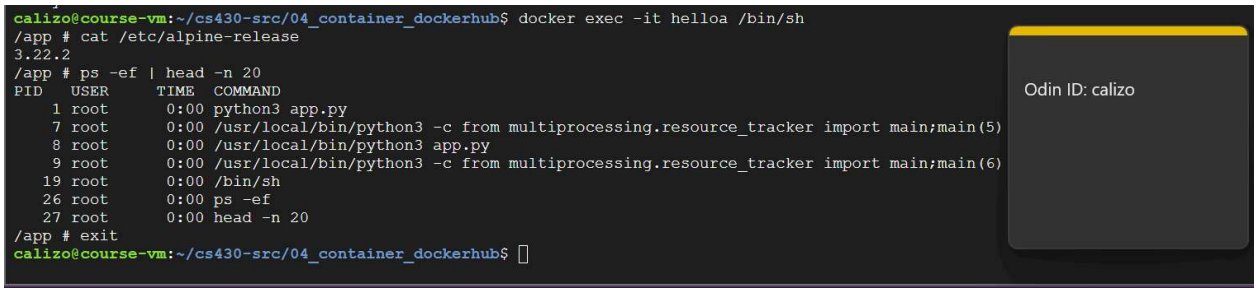
- docker exec -it helloa /bin/sh

granted successful access to the shell environment.

Inside, I confirmed the OS version and active processes:

- cat /etc/alpine-release
- ps -ef | head -n 20

2. Take a screenshot of the output of each.



The release file showed Alpine 3.22.2, and the process list verified that python3 app.py was running as PID 1, reflecting proper Flask execution under the root user.

After validation, I exited the container and cleaned up with:

- docker stop helloa
- docker rm helloa

Insight

This step illustrated the clear contrast between comprehensive and minimal container environments. Ubuntu offers completeness, while Alpine offers precision. Seeing the Guestbook run identically on both - yet with a tenth of the footprint - reaffirmed the strength of Docker’s abstraction: the same code, the same runtime, deployed with far less overhead.

Step 9: Docker Hub Alpine

For this step, I extended the containerization pipeline by publishing the helloalpine image to Docker Hub, enabling remote accessibility from any host. After logging in with docker login, I tagged the local image as jarrenx/helloalpine and pushed it to my public repository.

- docker tag helloalpine jarrenx/helloalpine
- docker push jarrenx/helloalpine

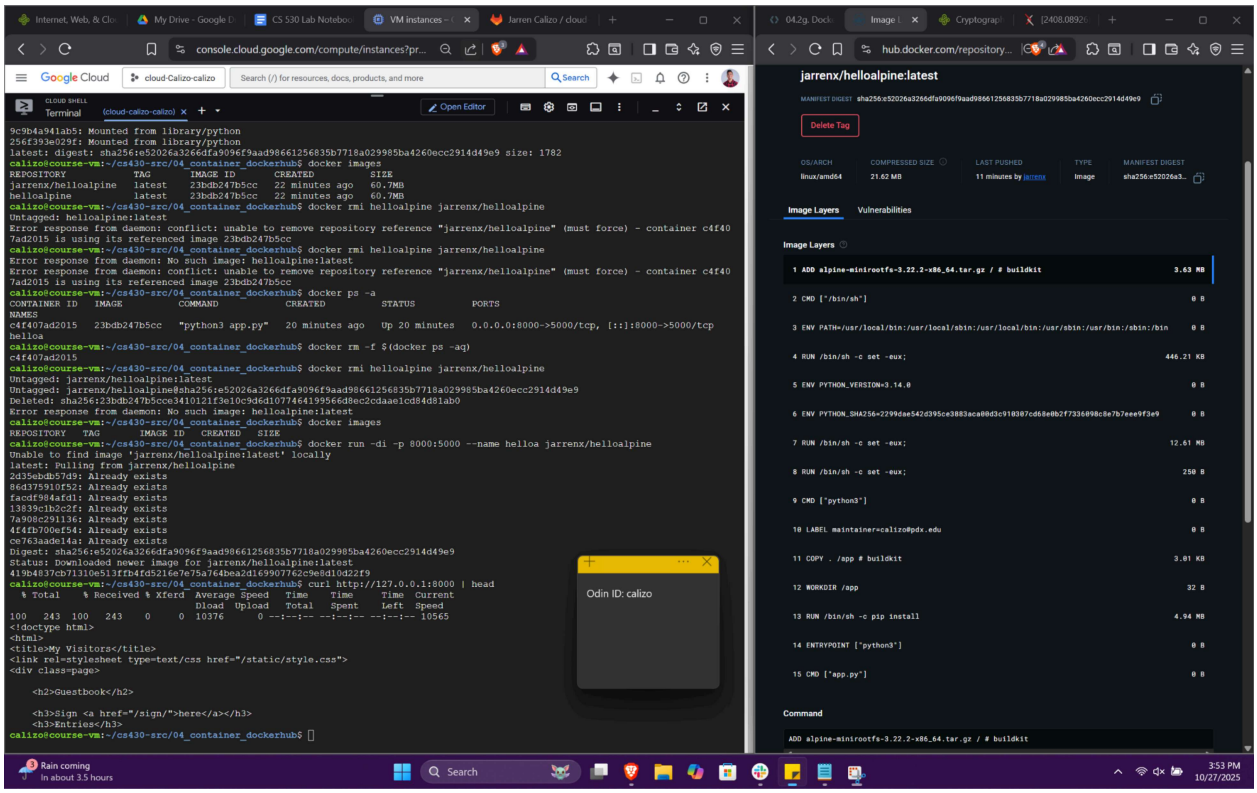
Once uploaded, I verified both local and remote copies with docker images, then deleted them using docker rmi helloalpine jarrenx/helloalpine to confirm the image could be pulled fresh from the registry.

Running it directly from Docker Hub with:

- docker run -di -p 8000:5000 --name helloa jarrenx/helloalpine

demonstrated how Docker retrieves and layers the image during deployment. Testing with curl http://127.0.0.1:8000 | head confirmed successful launch of the Guestbook service.

1. Take a screenshot of the container image and its size.



On Docker Hub, the jarrenx/helloalpine:latest repository displayed a compressed size of 21.62 MB. The largest contributing layer was the alpine-minirootfs (3.63 MB), representing the base operating system for the image. This validated Alpine’s compact design compared to the 501 MB Ubuntu build from earlier steps, highlighting its minimalism and deployment efficiency.

Step 10: Compute Engine Ubuntu VM Deployment

Next, I deployed the container on a remote Google Cloud Compute Engine VM to demonstrate Docker's portability across cloud environments. Using Cloud Shell, I launched the VM instance with Ubuntu 20.04 LTS using:

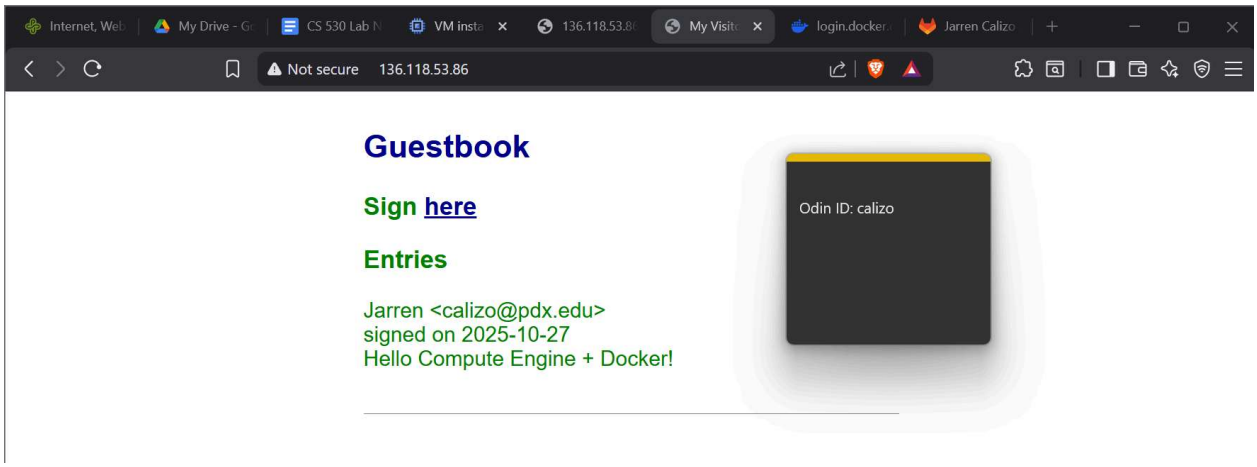
```
gcloud compute instances create docker-gb \
  --image-family=ubuntu-2004-lts \
  --image-project=ubuntu-os-cloud \
  --zone=us-west1-b \
  --machine-type=e2-micro \
  --tags=http-server,https-server
```

After connecting via SSH, I installed Docker, added my user to the docker group, and logged back in to enable privileges. The image was then pulled and run with:

- docker run -di -p 80:5000 --name helloa jarrenx/helloalpine

Navigating to the VM's External IP confirmed that the Guestbook application loaded seamlessly through the cloud-hosted container.

1. Take a screenshot of the entry that includes the VM's external IP address for your lab notebook.



I signed the Guestbook with the message "Hello Compute Engine + Docker!", verifying both HTTP access and proper write functionality at http://136.118.53.86/. The successful entry symbolized the full circle of container portability - developed locally, published globally, and deployed from the cloud with identical behavior.

Step 11: Clean Up

After confirming successful deployment, I deleted the VM instance to prevent idle resource usage and costs:

- gcloud compute instances delete docker-gb

This completed the lifecycle: build, tag, publish, deploy, and clean up - the hallmark of a well-managed DevOps container workflow.

Insight

These final steps elegantly encapsulated the promise of Docker: reproducibility and scale. From the minimal Alpine image to the cloud deployment on Compute Engine, every command reaffirmed how lightweight containers can unify local development and global deployment. Seeing the Guestbook serve requests from a public IP was both technically satisfying and conceptually grounding - proof that one image can truly run anywhere.